

Katholieke
Universiteit
Leuven

**Department of
Computer Science**

COMPUTER ARCHITECTUUR

Labo 5

Tano Pannekoucke (r0955339)

Academic year 2025–2026

Inhoudsopgave

1 Inleiding **2**

2 Implementatie en Methodologie **2**
2.1 Pinned Memory (Page-Locked Memory) 2
2.2 Streams en Events 2

3 Resultaten en Analyse **2**
3.1 Totale Executietijd 2
3.2 Schaalbaarheid en Speedup 3
3.3 Stabiliteit en Overhead (Boxplot Analyse) 4

4 Conclusie **5**

A Code **6**

1 Inleiding

In dit labo onderzoeken we het verschil tussen *Synchronous* en *Asynchronous* uitvoering binnen CUDA. Het doel is om computationele taken (reductie-operaties: som, product, minimum, maximum) uit te voeren op vier onafhankelijke arrays.

Bij een naïeve synchrone aanpak wacht de CPU tot de GPU klaar is voordat de volgende taak wordt voorbereid. Door gebruik te maken van *CUDA Streams* en asynchrone geheugenkopieën, proberen we de uitvoeringstijd te verkorten door CPU-werk (data generatie) te overlappen met GPU-werk (reductie). De resultaten zijn gebenchmarkt op een NVIDIA RTX 3050 met variërende datagrootte (1M tot 64M elementen) en thread-configuraties.

2 Implementatie en Methodologie

Om overlapping tussen CPU en GPU te realiseren, wijkt de implementatie op twee cruciale punten af van de standaard synchrone workflow.

2.1 Pinned Memory (Page-Locked Memory)

Een vereiste voor asynchrone `cudaMemcpyAsync`-operaties is het gebruik van *page-locked* (of pinned) geheugen op de host. In deze implementatie is daarom `cudaMallocHost` gebruikt in plaats van de standaard `cudaMalloc`.

Wanneer standaard *pageable* geheugen wordt gebruikt, moet de driver de data eerst naar een interne tijdelijke buffer kopiëren voordat de DMA-transfer kan starten. Dit blokkeert de CPU en verhindert asynchroniteit. Pinned memory stelt de GPU in staat om via DMA (Direct Memory Access) direct data te lezen vanuit het systeemgeheugen, terwijl de CPU parallel doorgaat met het uitvoeren van instructies.

2.2 Streams en Events

Om de taken onafhankelijk van elkaar te laten verlopen, maken we gebruik van vier `cudaStream_t` objecten. Elke operatie (Sum, Prod, Min, Max) wordt toegewezen aan een eigen stream. Commando's binnen één stream worden sequentieel uitgevoerd, maar commando's in verschillende streams kunnen gelijktijdig worden verwerkt door de GPU scheduler.

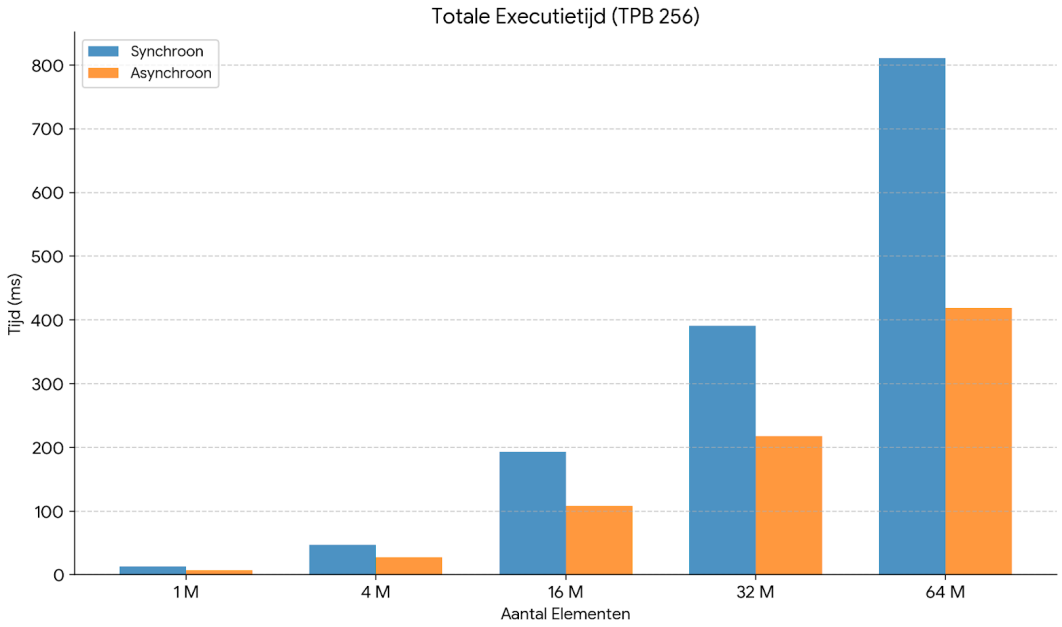
Voor de validatie van de tijdsmeting is gebruikgemaakt van `cudaEvent_t`. Standaard CPU-timers (zoals `std::chrono`) zijn ongeschikt binnen de asynchrone lus, omdat deze direct zouden terugkeren voordat de GPU-taak voltooid is.

3 Resultaten en Analyse

Er zijn 5 benchmark-runs uitgevoerd waarbij de datagrootte (N) varieerde van 1 miljoen tot 64 miljoen elementen en het aantal threads per block (TPB) varieerde tussen 128, 256 en 512.

3.1 Totale Executietijd

Figuur 1 toont het absolute verschil in milliseconden tussen de synchrone en asynchrone aanpak (bij TPB 256).

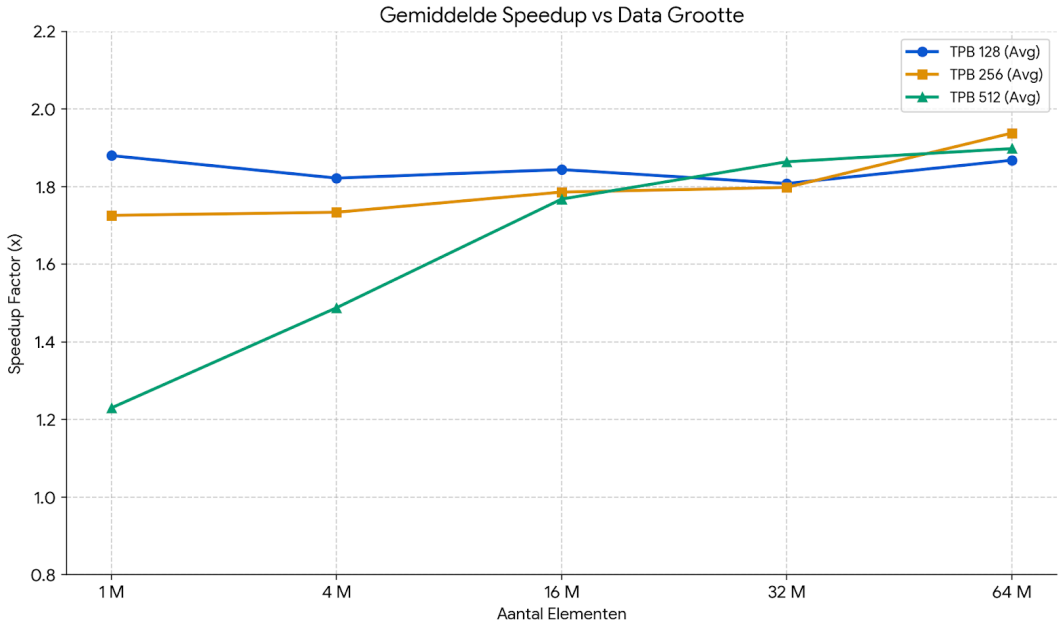


Figuur 1: Vergelijking van totale executietijd (Sync vs Async) bij TPB 256.

De grafiek toont een consistente halvering van de uitvoeringstijd bij de asynchrone methode. Bij $N = 64$ miljoen elementen daalt de tijd van ≈ 800 ms naar ≈ 420 ms. Dit bevestigt dat de pijplijn correct werkt: de totale tijd wordt niet langer bepaald door de som van CPU- en GPU-tijd, maar benadert het maximum van beiden.

3.2 Schaalbaarheid en Speedup

In Figuur 2 is de gemiddelde speedup uitgezet tegenover de datagrootte voor verschillende TPB-configuraties.



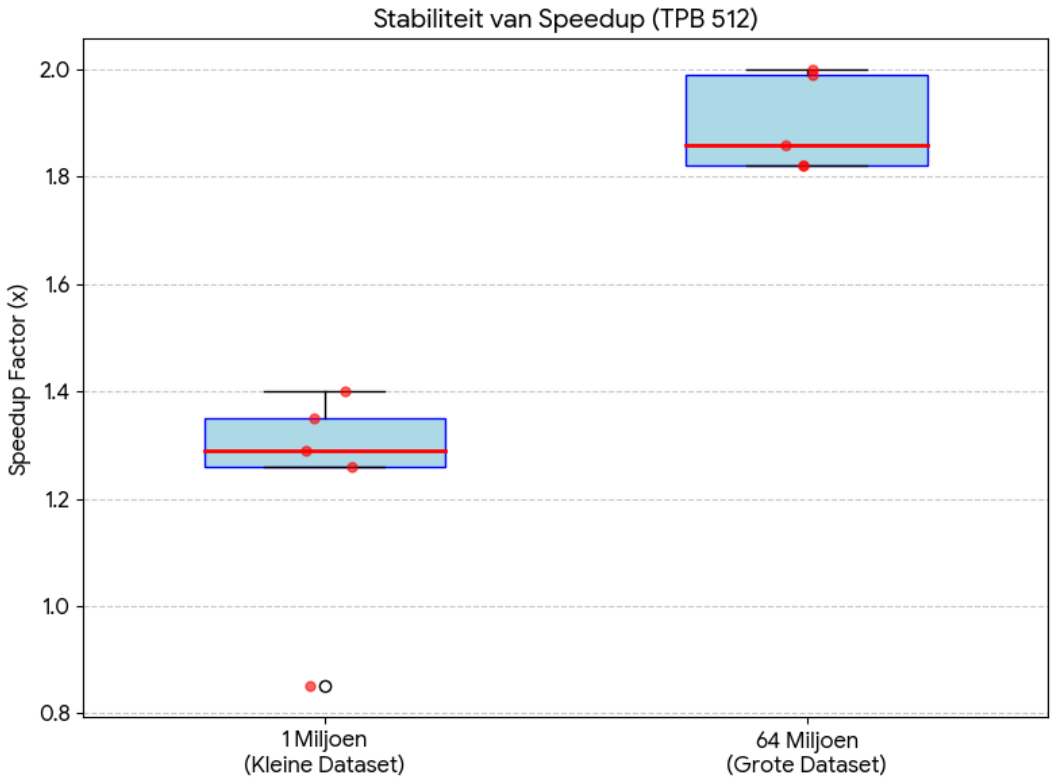
Figuur 2: Gemiddelde Speedup Factor in functie van datagrootte.

We observeren dat de speedup voor alle configuraties convergeert naar een factor van $\approx 1.9x$ bij grote datasets.

- De theoretische limiet is $2.0x$ (indien CPU- en GPU-tijd gelijk zijn en er perfecte overlap is).
- Het verlies van $0.1x$ wordt verklaard door overhead van de PCIe-bus, kernel launch latency en het feit dat de CPU-taak (`fillData`) en GPU-taak (`kernelReduce`) niet exact even lang duren.

3.3 Stabiliteit en Overhead (Boxplot Analyse)

Een belangrijk fenomeen doet zich voor bij de kleinere datasets. Figuur 3 toont de spreiding van de speedup over de 5 runs voor TPB 512.



Figuur 3: Boxplot van de Speedup bij TPB 512: onstabiliteit bij lage N.

Aan de linkerzijde (1 miljoen elementen) is een grote variantie zichtbaar, met in één geval zelfs een speedup lager dan $1.0x$ (trager dan synchroon).
Verklaring: Dit wordt veroorzaakt door overhead.

- Bij een kleine dataset (1M) is de GPU-berekening zeer snel voltooid.
- De tijd die nodig is om streams aan te maken, events te recorden en de kernels te lanceren is in verhouding groot.
- TPB 512 lanceert minder blocks dan TPB 128, wat bij kleine data kan leiden tot een lagere *occupancy* (niet alle SM's op de RTX 3050 zijn volledig verzadigd).
- Hierdoor weegt de administratieve kost van asynchroniteit zwaarder dan de winst door overlap.

Bij 64 miljoen elementen (rechterzijde) is de spreiding minimaal en ligt de speedup stabiel rond de $1.9x$. Hier is de rekentijd groot genoeg om de overhead volledig te maskeren.

4 Conclusie

Uit de experimenten blijkt dat *Asynchronous Operation* een aanzienlijke snelheidswinst oplevert voor data-intensieve applicaties. Door gebruik te maken van `cudaMallocHost` en Streams hebben we een speedup van circa $1.9x$ gerealiseerd op de RTX 3050.

De analyse toont echter aan dat asynchroon programmeren overhead introduceert: bij kleine workloads domineert deze overhead, wat kan leiden tot instabiele prestaties. Voor optimale prestaties moet de workload per kernel groot genoeg zijn om de latentie van streams en geheugentransfers te compenseren.

Referenties

[1] Feys, T. (2024). *Geavanceerde Computerarchitectuur Session 5: Dynamic parallelism and asynchronous operation*. KU Leuven.

[2] NVIDIA Corporation. (2024). *CUDA C++ Programming Guide*.

[3] Google. (2025). *Gemini* . `https://gemini.google.com`

A Code

Hieronder bevindt zich de volledige C++/CUDA implementatie gebruikt voor dit labo.

```

#include <cstdio>
#include <cstdlib>
#include <vector>
#include <iostream>
#include <iomanip>  // Nodig voor de tabel uitlijning (setw)
#include <cuda_runtime.h>
#include <climits>

#define CUDA_CHECK(x) do { cudaError_t e=(x); if(e!=cudaSuccess){ \
  fprintf(stderr,"CUDA ERROR %s:%d: %s\n",__FILE__,__LINE__,cudaGetErrorString(e)); \
  std::exit(EXIT_FAILURE);} } while(0)

enum OpType { OP_SUM, OP_PROD, OP_MIN, OP_MAX };

// --- Hulpfuncties ---
int my_min(int a, int b) { return (a < b) ? a : b; }

// ------------------------------------------------------------------------
// 1. DEVICE KERNELS
// ------------------------------------------------------------------------

__device__ __forceinline__ int getIdentity(OpType op) {
    switch(op) {
        case OP_SUM: return 0;
        case OP_PROD: return 1;
        case OP_MIN: return INT_MAX;
        case OP_MAX: return INT_MIN;
        default: return 0;
    }
}

__device__ __forceinline__ int calculate(OpType op, int a, int b) {
    switch(op) {
        case OP_SUM: return a + b;
        case OP_PROD: return a * b;
        case OP_MIN: return (a < b) ? a : b;
        case OP_MAX: return (a > b) ? a : b;
        default: return a;
    }
}

template <OpType OP>
__global__ void kernelReduce(const int* d_in, int* d_out, int N) {
    extern __shared__ int sdata[];
    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    int localVal = getIdentity(OP);
    for (int i = idx; i < N; i += blockDim.x * gridDim.x) {
        localVal = calculate(OP, localVal, d_in[i]);
    }

    sdata[tid] = localVal;
    __syncthreads();

    for (int stride = blockDim.x >> 1; stride > 0; stride >>= 1) {
        if (tid < stride) {
```

```
            sdata[tid] = calculate(OP, sdata[tid], sdata[tid + stride]);
        }
        __syncthreads();
    }

    if (tid == 0) d_out[blockIdx.x] = sdata[0];
}

void launchKernel(OpType op, const int* d_in, int* d_out, int N, int blocks, int tpb, int shmem, cudaStream_t stream) {
    switch(op) {
        case OP_SUM:  kernelReduce<OP_SUM><<<blocks, tpb, shmem, stream>>>(d_in, d_out, N); break;
        case OP_PROD: kernelReduce<OP_PROD><<<blocks, tpb, shmem, stream>>>(d_in, d_out, N); break;
        case OP_MIN:  kernelReduce<OP_MIN><<<blocks, tpb, shmem, stream>>>(d_in, d_out, N); break;
        case OP_MAX:  kernelReduce<OP_MAX><<<blocks, tpb, shmem, stream>>>(d_in, d_out, N); break;
    }
}

// ------------------------------------------------------------------------
// 2. HOST HELPERS & BENCHMARKS
// ------------------------------------------------------------------------

void fillData(int* data, int N, OpType op) {
    for (int i = 0; i < N; i++) {
        if (op == OP_PROD) data[i] = 1;
        else if (op == OP_SUM) data[i] = 1;
        else data[i] = (i % 1000);
    }
}

float runSyncBenchmark(int N, int tpb) {
    int blocks = my_min(65535, (N + tpb - 1) / tpb);
    int* h_in = new int[N];
    int* d_in;       CUDA_CHECK(cudaMalloc(&d_in, N * sizeof(int)));
    int* d_partial;  CUDA_CHECK(cudaMalloc(&d_partial, blocks * sizeof(int)));
    int* d_result;   CUDA_CHECK(cudaMalloc(&d_result, sizeof(int)));
    int h_result;
    OpType ops[] = {OP_SUM, OP_PROD, OP_MIN, OP_MAX};

    cudaEvent_t start, stop;
    cudaEventCreate(&start); cudaEventCreate(&stop);

    cudaEventRecord(start);

    for (int i = 0; i < 4; i++) {
        fillData(h_in, N, ops[i]);
        CUDA_CHECK(cudaMemcpy(d_in, h_in, N * sizeof(int), cudaMemcpyHostToDevice));
        launchKernel(ops[i], d_in, d_partial, N, blocks, tpb, tpb*sizeof(int), 0);
        launchKernel(ops[i], d_partial, d_result, blocks, 1, tpb, tpb*sizeof(int), 0);
        CUDA_CHECK(cudaMemcpy(&h_result, d_result, sizeof(int), cudaMemcpyDeviceToHost));
    }

    cudaEventRecord(stop);
    cudaEventSynchronize(stop);

    float ms = 0;
    cudaEventElapsedTime(&ms, start, stop);

    cudaFree(d_in); cudaFree(d_partial); cudaFree(d_result);
    delete[] h_in;
    cudaEventDestroy(start); cudaEventDestroy(stop);
    return ms;
}

float runAsyncBenchmark(int N, int tpb) {
    int blocks = my_min(65535, (N + tpb - 1) / tpb);
    int* h_in[4]; int* d_in[4]; int* d_partial[4]; int* d_result[4];
    cudaStream_t streams[4];
    OpType ops[] = {OP_SUM, OP_PROD, OP_MIN, OP_MAX};
```

```
127
128     for(int i=0; i<4; i++) {
129         CUDA_CHECK(cudaMallocHost(&h_in[i], N * sizeof(int)));
130         CUDA_CHECK(cudaMalloc(&d_in[i], N * sizeof(int)));
131         CUDA_CHECK(cudaMalloc(&d_partial[i], blocks * sizeof(int)));
132         CUDA_CHECK(cudaMalloc(&d_result[i], sizeof(int)));
133         CUDA_CHECK(cudaStreamCreate(&streams[i]));
134     }
135
136     cudaEvent_t start, stop;
137     cudaEventCreate(&start); cudaEventCreate(&stop);
138
139     cudaEventRecord(start);
140
141     for (int i = 0; i < 4; i++) {
142         fillData(h_in[i], N, ops[i]);
143         CUDA_CHECK(cudaMemcpyAsync(d_in[i], h_in[i], N * sizeof(int), cudaMemcpyHostToDevice,
      streams[i]));
144         launchKernel(ops[i], d_in[i], d_partial[i], N, blocks, tpb, tpb*sizeof(int), streams[
    i]);
145         launchKernel(ops[i], d_partial[i], d_result[i], blocks, 1, tpb, tpb*sizeof(int),
    streams[i]);
146         CUDA_CHECK(cudaMemcpyAsync(h_in[i], d_result[i], sizeof(int), cudaMemcpyDeviceToHost,
      streams[i]));
147     }
148
149     cudaDeviceSynchronize();
150
151     cudaEventRecord(stop);
152     cudaEventSynchronize(stop);
153
154     float ms = 0;
155     cudaEventElapsedTime(&ms, start, stop);
156
157     for(int i=0; i<4; i++) {
158         cudaFreeHost(h_in[i]); cudaFree(d_in[i]);
159         cudaFree(d_partial[i]); cudaFree(d_result[i]);
160         cudaStreamDestroy(streams[i]);
161     }
162     cudaEventDestroy(start); cudaEventDestroy(stop);
163     return ms;
164 }
165
166 // ---------------------------------------------------------------------------
167 // 3. MAIN (TABEL OUTPUT)
168 // ---------------------------------------------------------------------------
169 int main() {
170     std::vector<int> sizes = {
171         1024 * 1024,      // 1M
172         4 * 1024 * 1024,  // 4M
173         16 * 1024 * 1024, // 16M
174         32 * 1024 * 1024, // 32M
175         64 * 1024 * 1024  // 64M
176     };
177
178     std::vector<int> tpbs = { 128, 256, 512 };
179
180     // Warm-up
181     { int* tmp; cudaMalloc(&tmp, sizeof(int)); cudaFree(tmp); }
182
183     // --- Header ---
184     std::cout << "-------------------------------------------------------------------------\
    n";
185     std::cout << "|   Elements | Threads |    Sync Time |   Async Time |   Speedup |\n";
186     std::cout << "-------------------------------------------------------------------------\
    n";
187
188     for (int tpb : tpbs) {
189         for (int N : sizes) {
190             float t_sync = runSyncBenchmark(N, tpb);
191             cudaDeviceSynchronize();
192             float t_async = runAsyncBenchmark(N, tpb);
193             float speedup = t_sync / t_async;
```

```
194
195             // --- Row Output ---
196             std::cout << "| " << std::setw(10) << N
197                       << " | " << std::setw(7) << tpb
198                       << " | "
199                       << std::fixed << std::setprecision(2) << std::setw(9) << t_sync << " ms
    "
200                       << " | "
201                       << std::fixed << std::setprecision(2) << std::setw(9) << t_async << "
    ms"
202                       << " | "
203                       << std::fixed << std::setprecision(2) << std::setw(7) << speedup << " x
    "
204                       << " |\n";
205         }
206         std::cout << "
    ------------------------------------------------------------------------------\n";
207     }
208
209     return 0;
210 }
```